



BITS Pilani
K K Birla Goa Campus

ARCHITECTURE

MICROPROCESSORS & INTERFACING (CS F241)

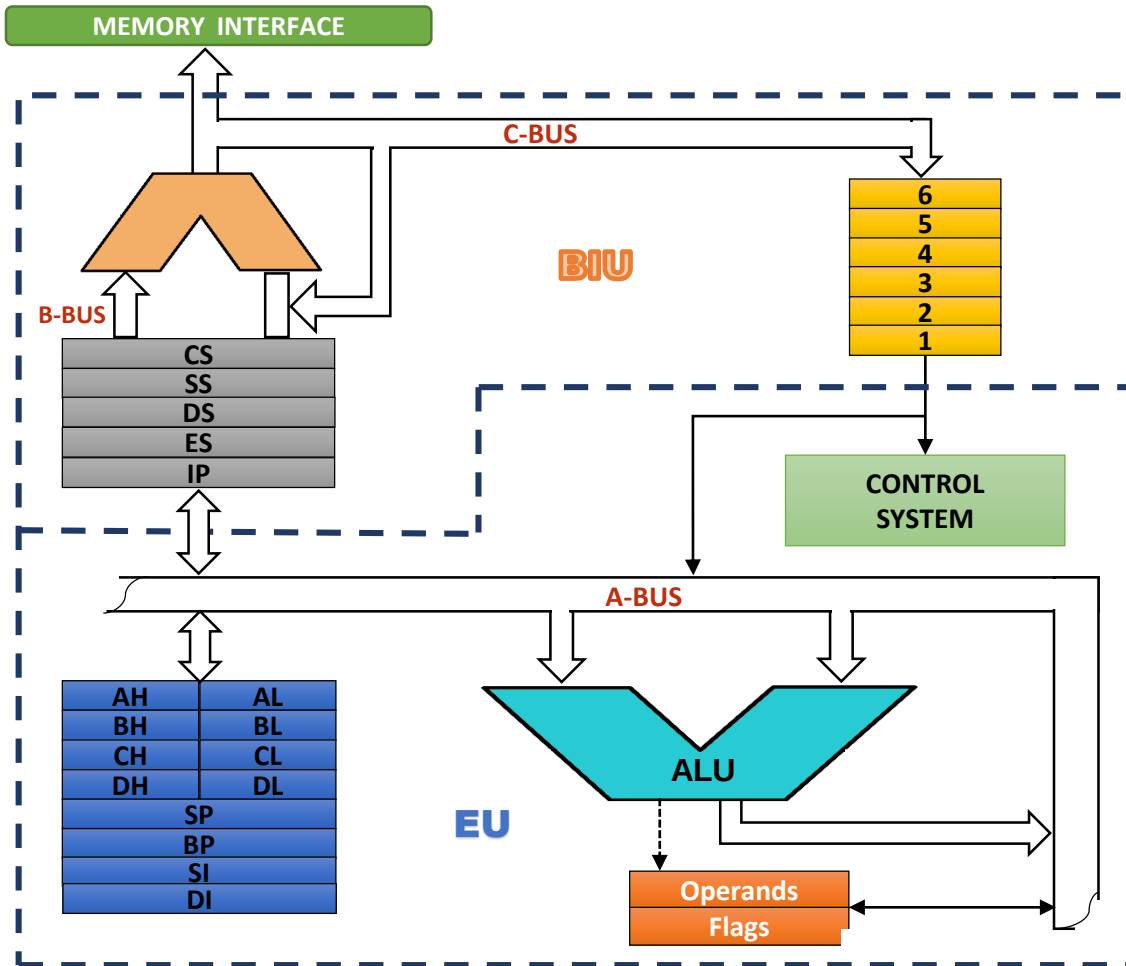
Dr. Gargi Alavani
Department of CS & IS



The 8086 Microprocessor

- First 16-bit microprocessor released in 1978
- 20-bit address bus, 1,048,576 memory locations
- 16-bit data bus, read or write 16 bits or 8 bits at a time
- Segmentation of memory for increasing execution speed
- Early implementation of pipelining

Block Diagram of 8086



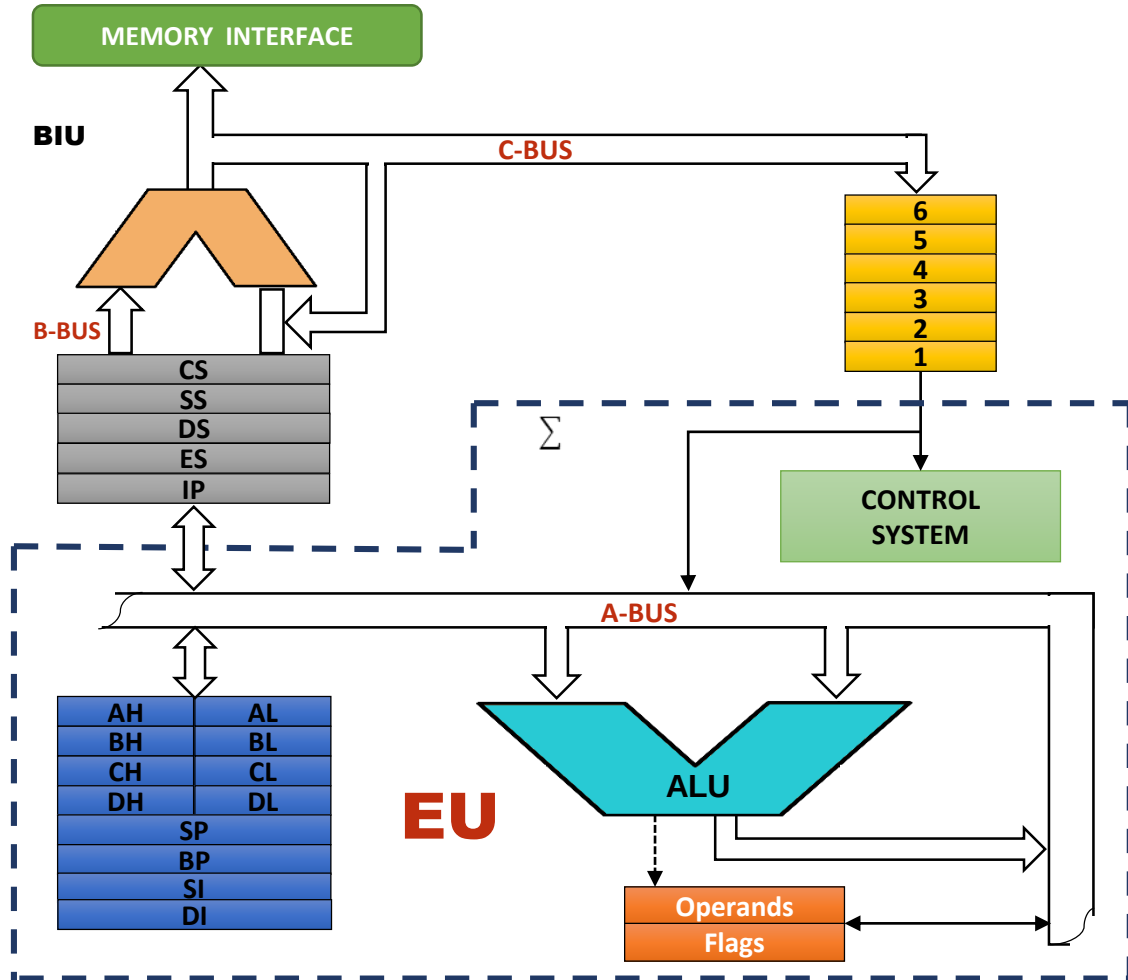
8086 CPU is divided into two parts:

- Bus Interface Unit (BIU)
- Execution Unit (EU)

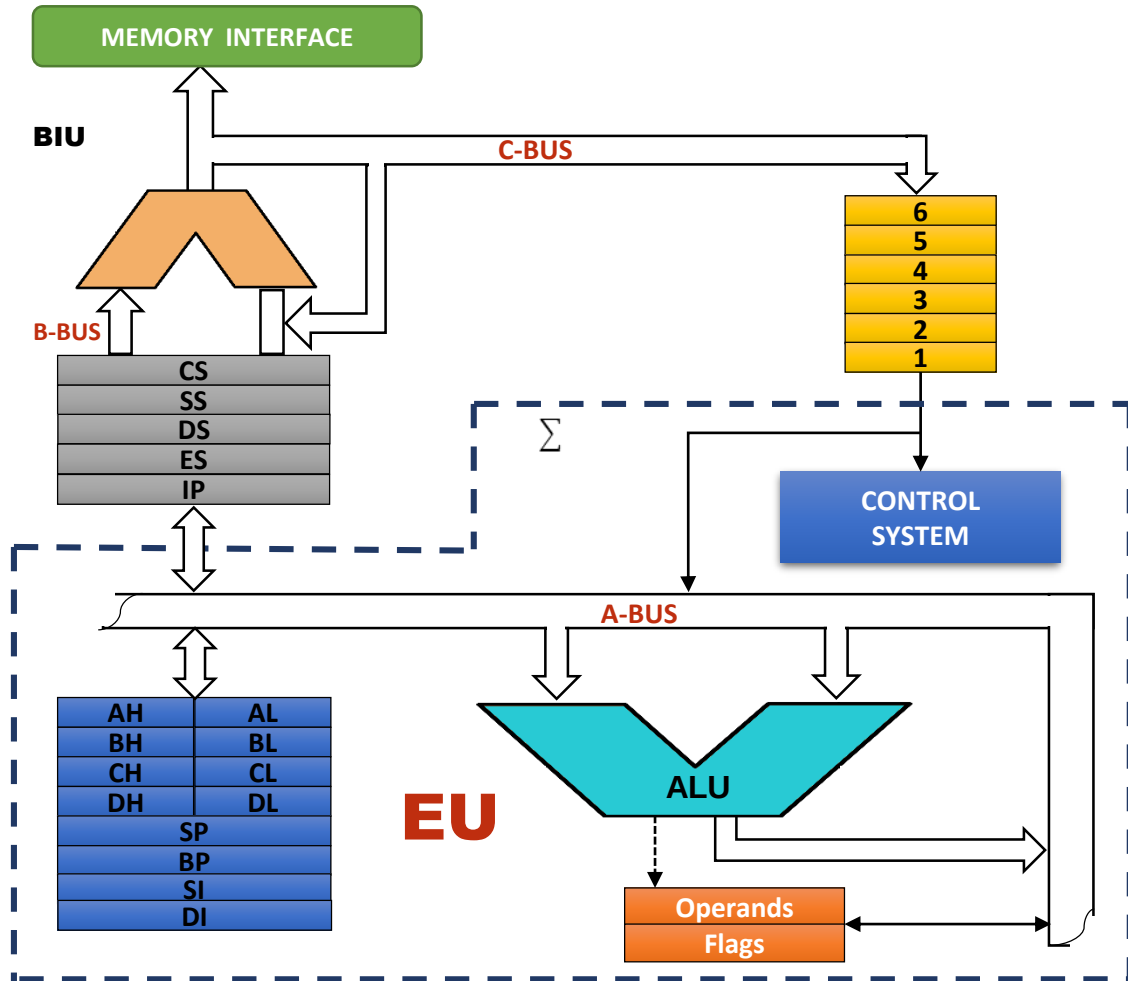
Bus Interface Unit (BIU) handles all transfers of data and addresses on the buses for EU

Execution Unit (EU) tells BIU where to fetch instructions or data from, decodes and executes instruction.

Execution Unit (EU)



Execution Unit (EU)

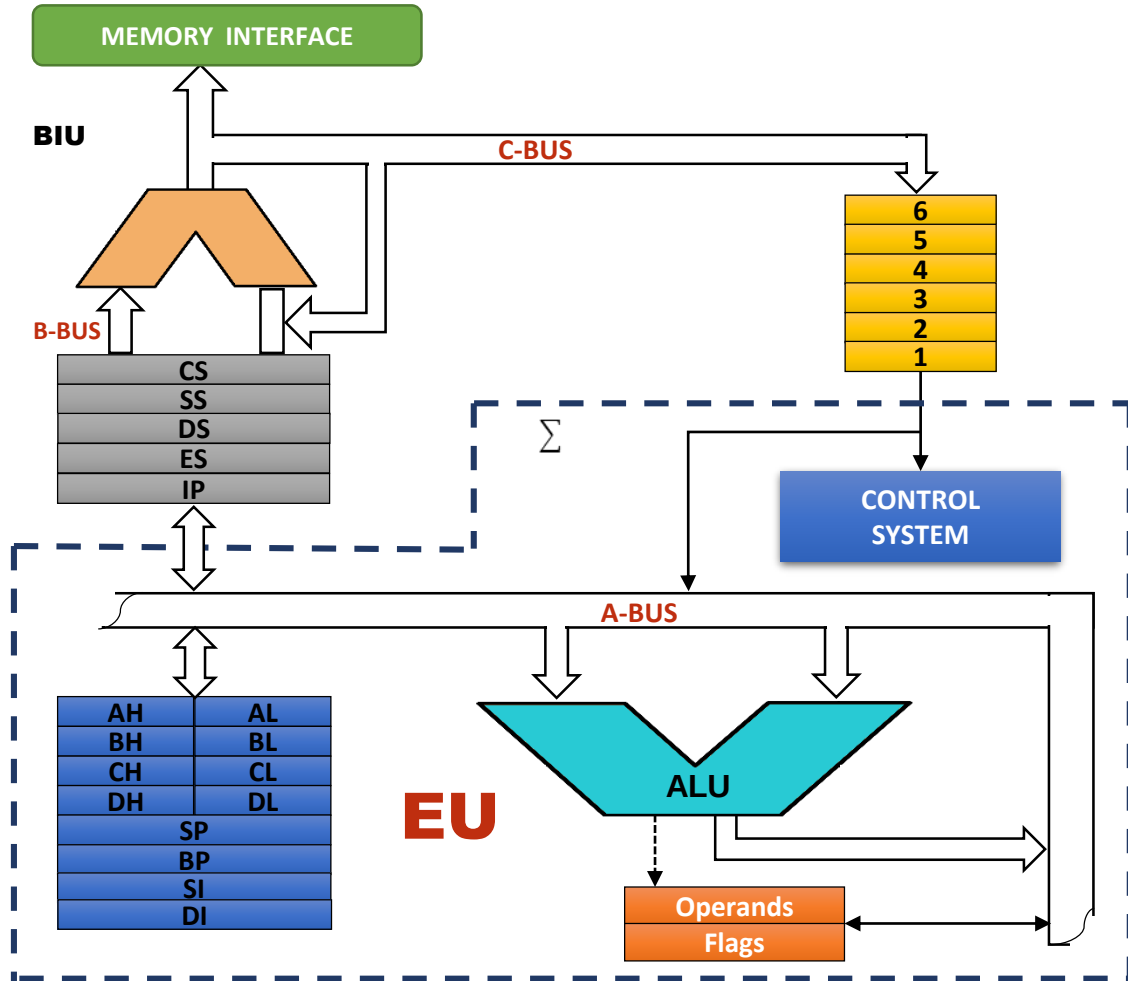


In control system, Decoder translates instructions fetched from memory

Control system directs internal operations using control signal:

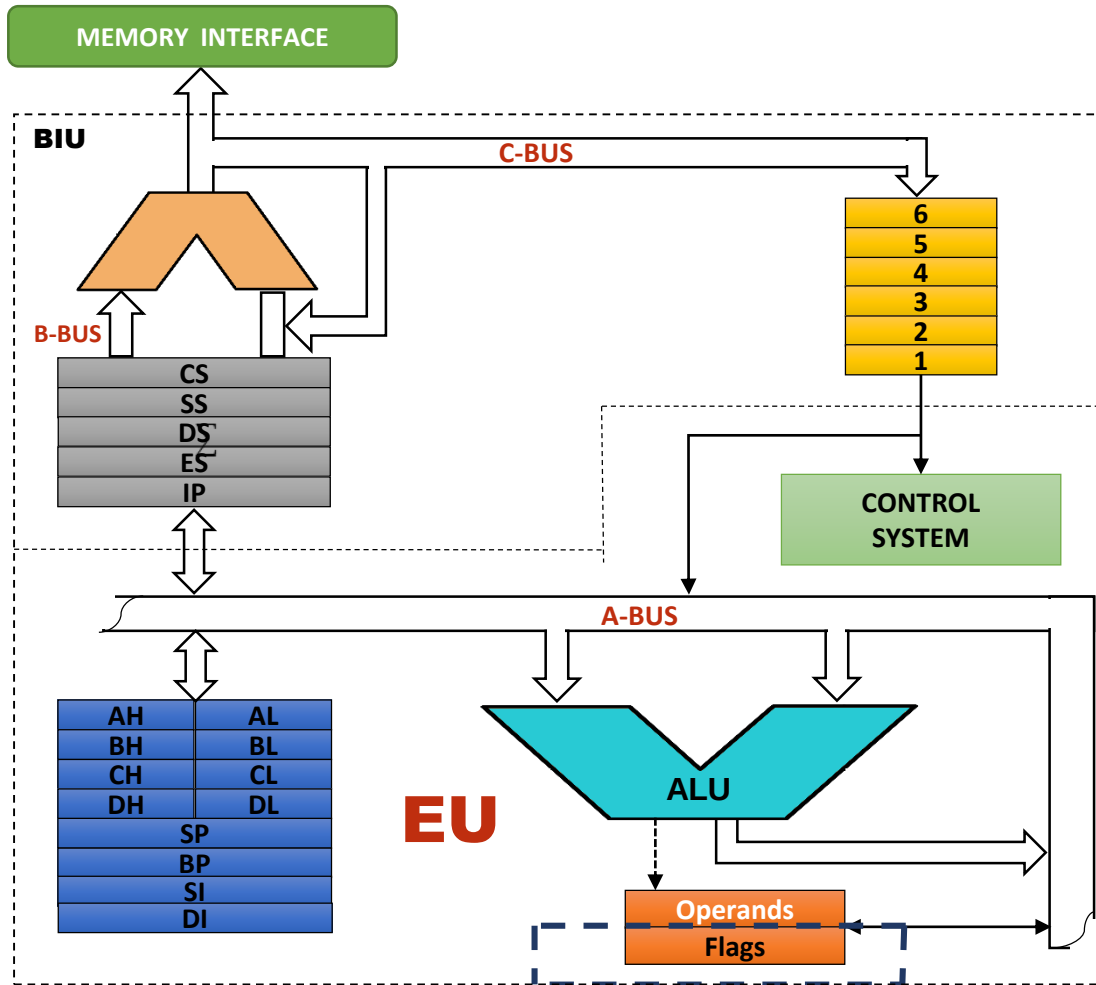
- RD/WR
- M/IO
- INTR/INTA

Execution Unit (EU)



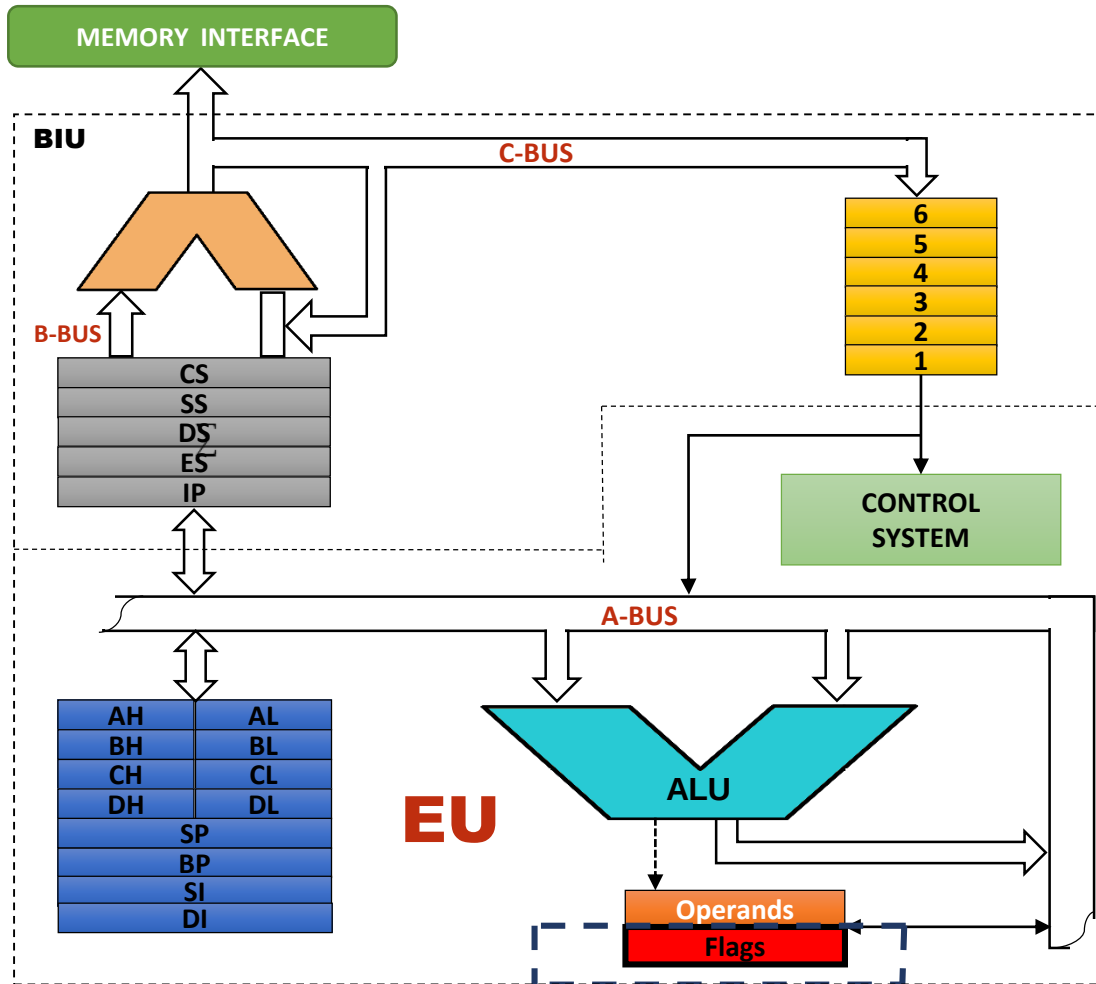
16 bit ALU performs operations such as **ADD, SUB, AND, OR, XOR**

Execution Unit (EU)



FLAG is a flip-flop which indicates some condition produced by the Execution Unit

Execution Unit (EU)



Conditional Flag Registers

- Carry flag (CF)
- Parity flag (PF)
- Auxiliary carry flag (AF)
- Zero flag (ZF)
- Sign flag (SF)
- Overflow flag (OF)

Control Flag Registers

- Trap flag (TF)
- Interrupt flag (IF)
- Direction flag (DF)

Example of Conditional Flag Registers

```
  0011 0100 1101 1100
+0000 0111 0010 1110
-----
  0011 1100 0000 1010
```

CF = 0

PF = 1

AF = 1

ZF = 0

SF = 0

OF = 0

- Carry flag (CF) - Carry/borrow out of MSB occurs
- Parity flag (PF) - set to 1 if low-order 8 bits (low order byte) contain even number of 1's
- Auxiliary carry flag (AF) - carry out of bit 3
- Zero flag (ZF) - set to 1 if result is 0; set to 0 if result is nonzero
- Sign flag (SF) - MSB of result
- Overflow flag (OF) - Set (1) when there is a mismatch between the carry into and carry out of the most significant bit (MSB) in signed arithmetic.

Example 2

AX: 0xAAAA

(Binary: 1010 1010 1010 1010)

BX: 0xF557

(Binary: 1111 0101 0101 0111)

Operation: ADD AX, BX

- Carry flag (CF) - Carry/borrow out of MSB occurs
- Parity flag (PF) - set to 1 if low-order 8 bits (low order byte) contain even number of 1's
- Auxiliary carry flag (AF) - carry out of bit 3
- Zero flag (ZF) - set to 1 if result is 0; set to 0 if result is nonzero
- Sign flag (SF) - MSB of result
- Overflow flag (OF) - Set (1) when there is a mismatch between the carry into and carry out of the most significant bit (MSB) in signed arithmetic.

Example 2

AX: 0xAAAA

(Binary: 1010 1010 1010 1010)

BX: 0xF557

(Binary: 1111 0101 0101 0111)

Operation: ADD AX, BX

CF = 1

OF = 0

AF = 1

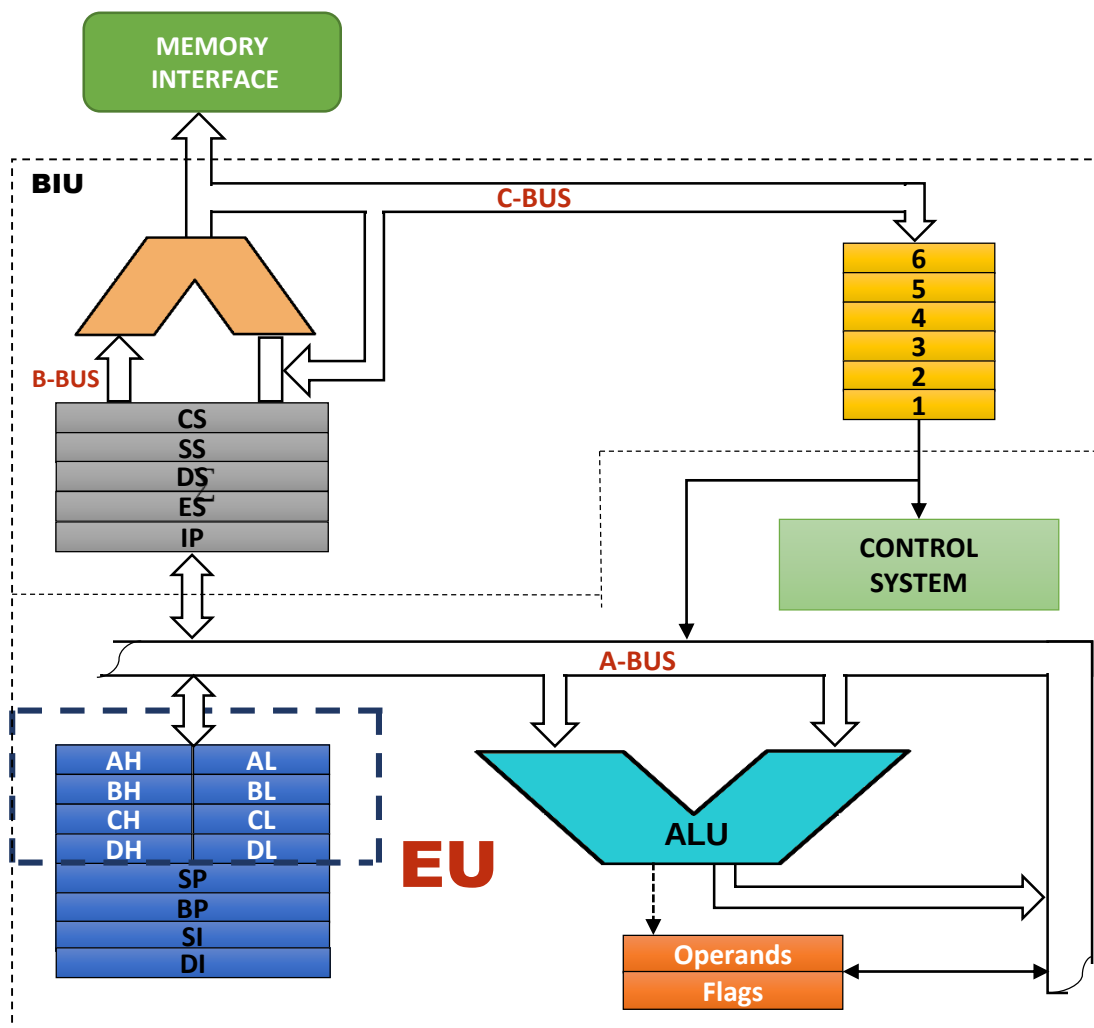
ZF = 0

SF = 1

PF = 0

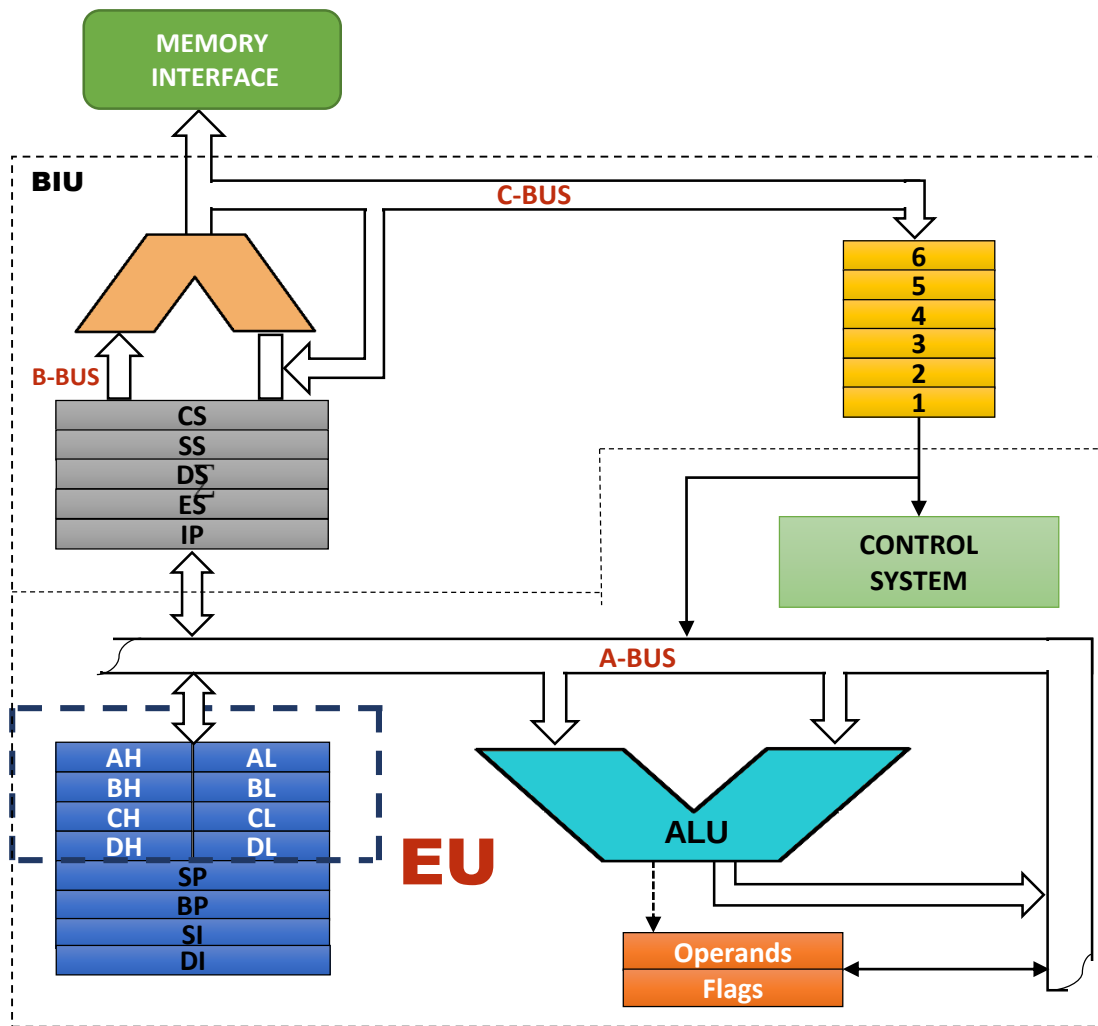
- Carry flag (CF) - Carry/borrow out of MSB occurs
- Parity flag (PF) - set to 1 if low-order 8 bits (low order byte) contain even number of 1's
- Auxiliary carry flag (AF) - carry out of bit 3
- Zero flag (ZF) - set to 1 if result is 0; set to 0 if result is nonzero
- Sign flag (SF) - MSB of result
- Overflow flag (OF) - Set (1) when there is a mismatch between the carry into and carry out of the most significant bit (MSB) in signed arithmetic.

Execution Unit (EU)



General Purpose Registers

- Eight general purpose 8-bit Registers
- Can be used individually to store data
- Certain pairs can be used together to store 16-bit data words.
- AL is an accumulator for arithmetic and logic operations.



Execution Unit (EU)

AX (accumulator register)

Preferred for arithmetic, logic, and I/O operations.

BX (base register)

Preferred for arithmetic, logic, and I/O operations.

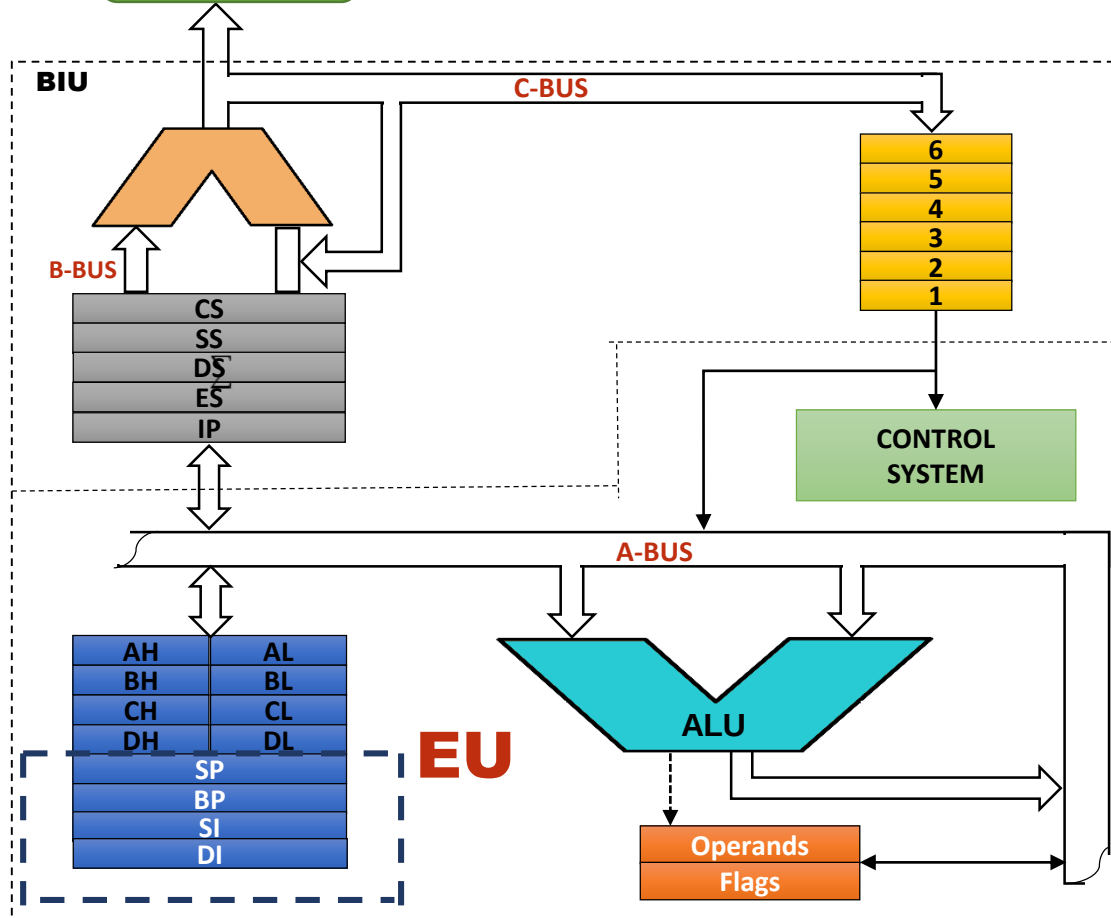
CX (count register)

Acts as a counter for loops and shift/rotate instructions.

DX (data register)

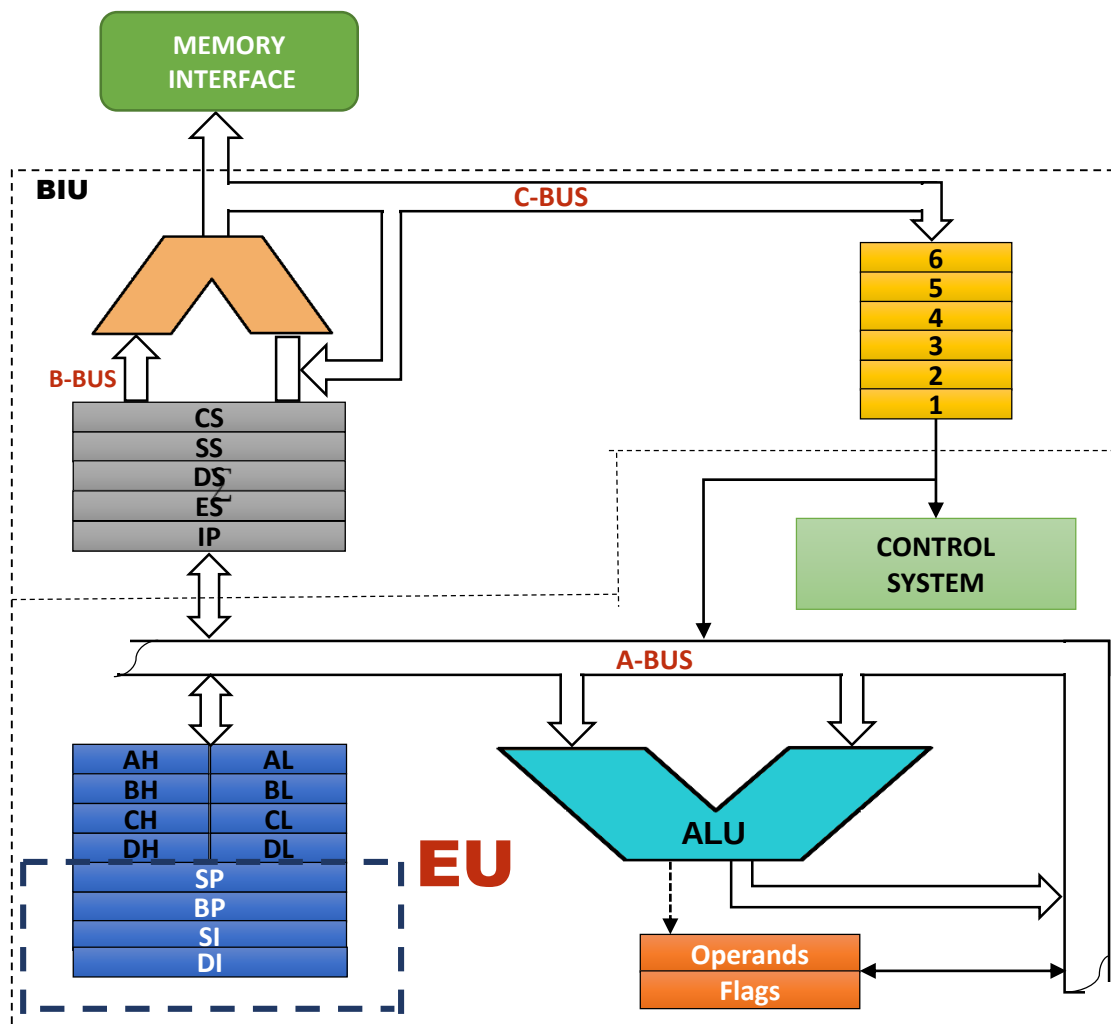
Used in multiplication/division and for I/O port addressing.

Execution Unit (EU)



Pointer & Index Registers

- 64KB Segment is set aside as a stack which stores addresses and data while subprogram is executing.
- Physical address(PA) of stack = $SS + SP$
- Three more registers: SI, DI and BP which can be used as temporary storage of data
- Main use is to hold 16-bit offset of a data word
e.g. $PA = DS \times 10H + SI$



Execution Unit (EU)

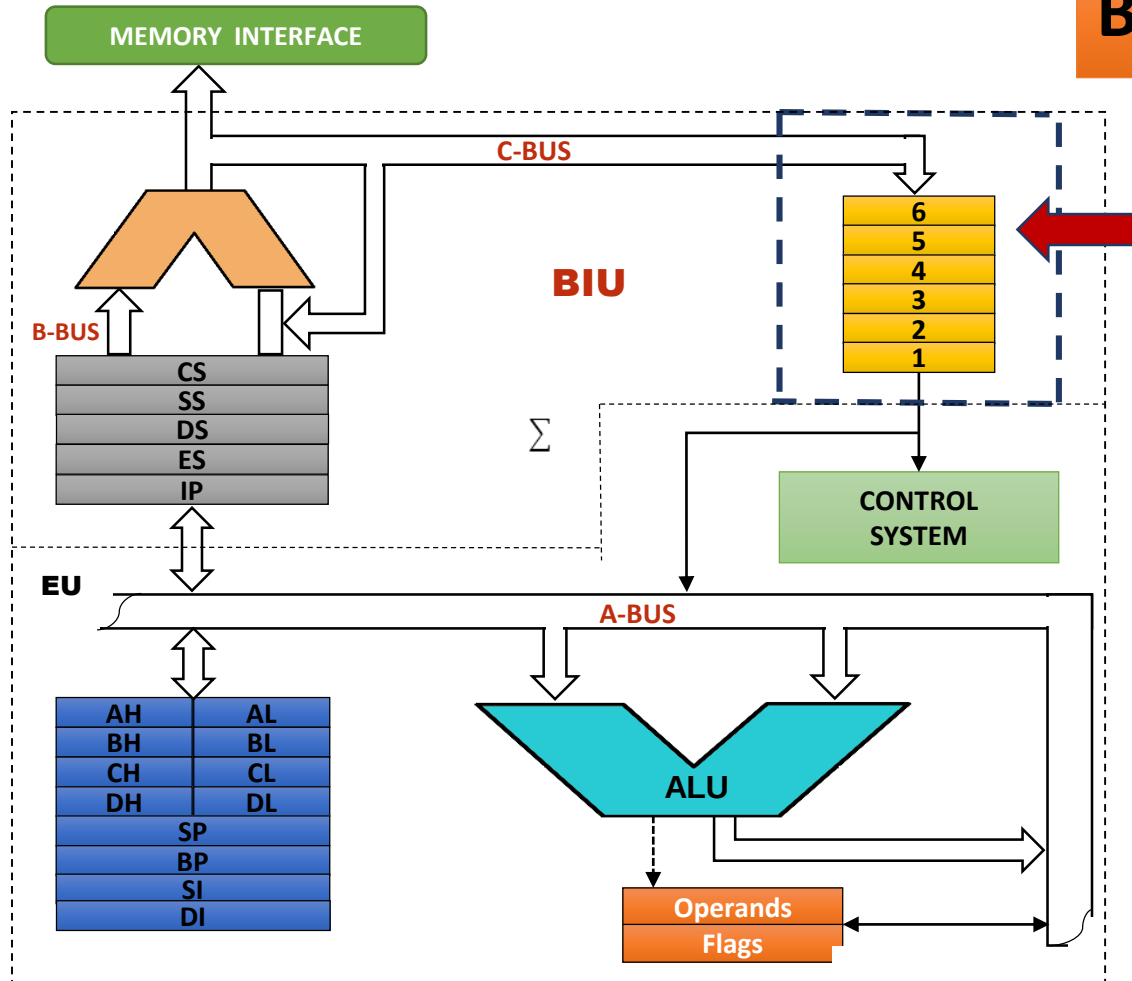
SP (Stack Pointer register): used as a stack pointer for push and pop operations, and for accessing function call frames and local variables.

BP (Base Pointer register): used as a base pointer for stack operations and for accessing function parameters and local variables.

SI (Source Index register): used as a source pointer for memory operations, and is also used in indexed addressing modes.

DI (Destination Index register): used as a destination pointer for memory operations, and is also used in indexed addressing modes.

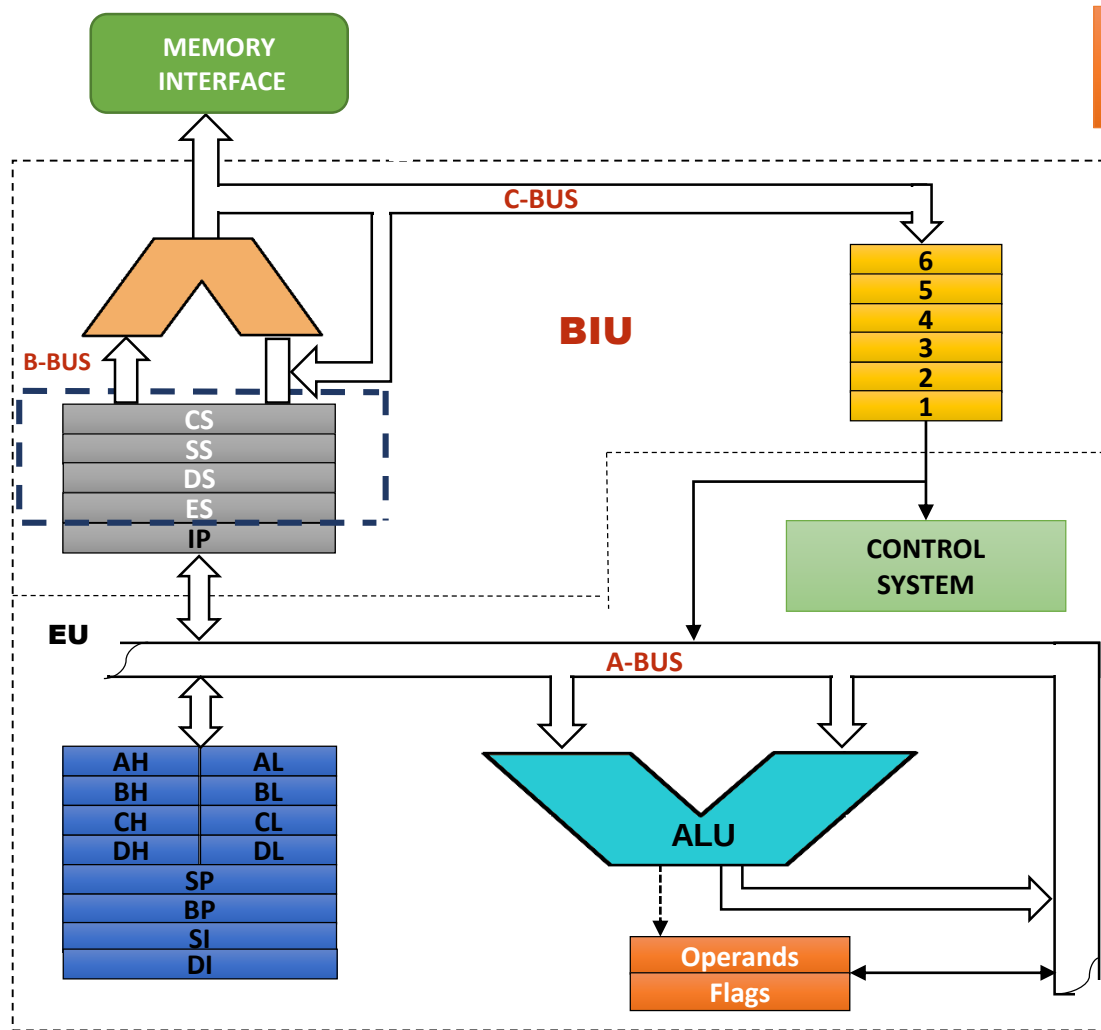
Bus Interface Unit (BIU)



Instruction Queue

- BIU fetches up to **six instruction** bytes and stores in a queue
- Process of fetching next instruction while executing current instruction is called **pipelining**
- Speeds up the process except for **JMP** and **CALL**

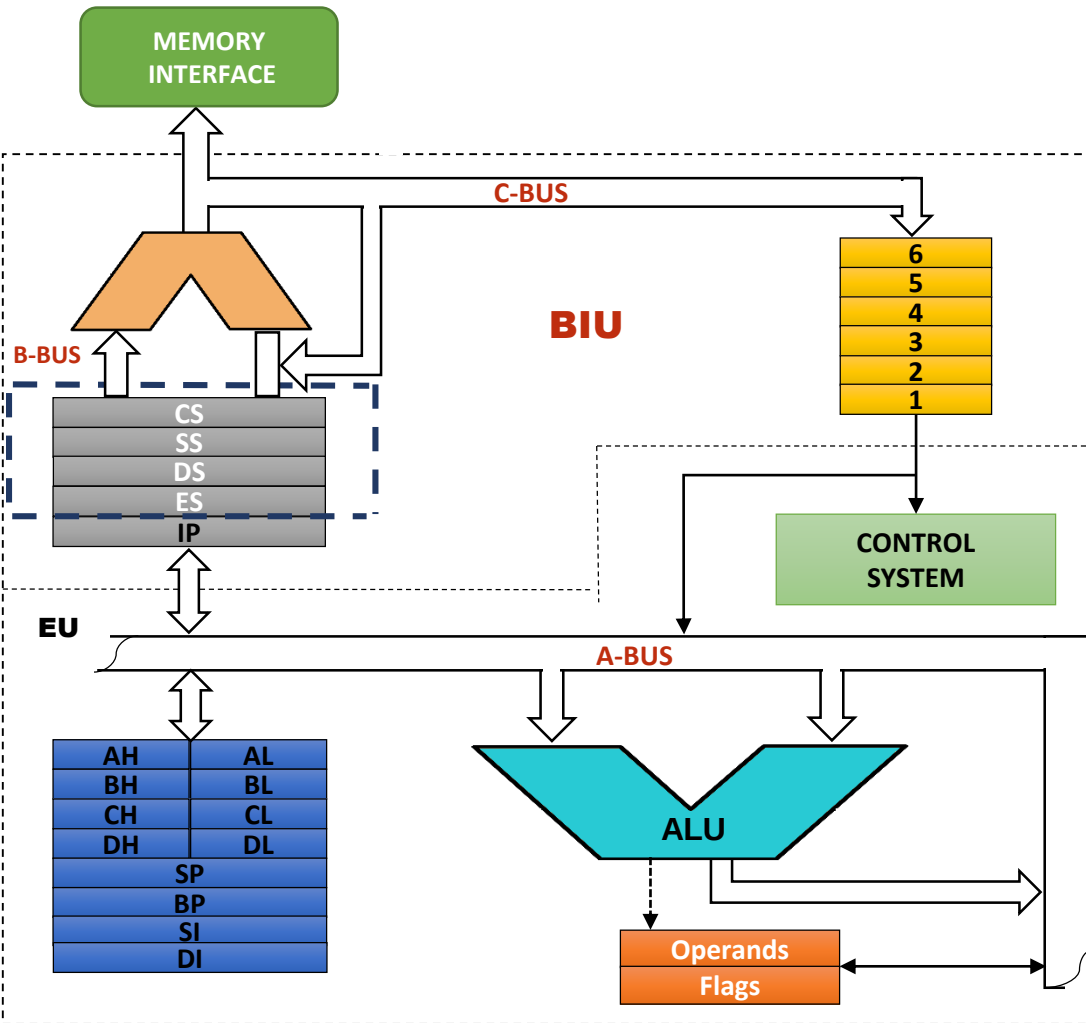
Bus Interface Unit (BIU)



The Segment Registers

- BIU sends out 20-bit addresses so it can address any of 2^{20} or 1,048,576 bytes (1MB) in memory.
- Four segment registers hold the upper 16 bits of the starting address of four memory segments
- The effective address calculation ($\text{segment_register} \times 10H + \text{offset}$) allows addressing up to 1 MB of memory (2^{20}).

Bus Interface Unit (BIU)



CS (Code Segment register): holds the starting address of the code segment where the currently executing program is located.

SS (Stack Segment register): holds the starting address of the stack segment where the program's stack is located.

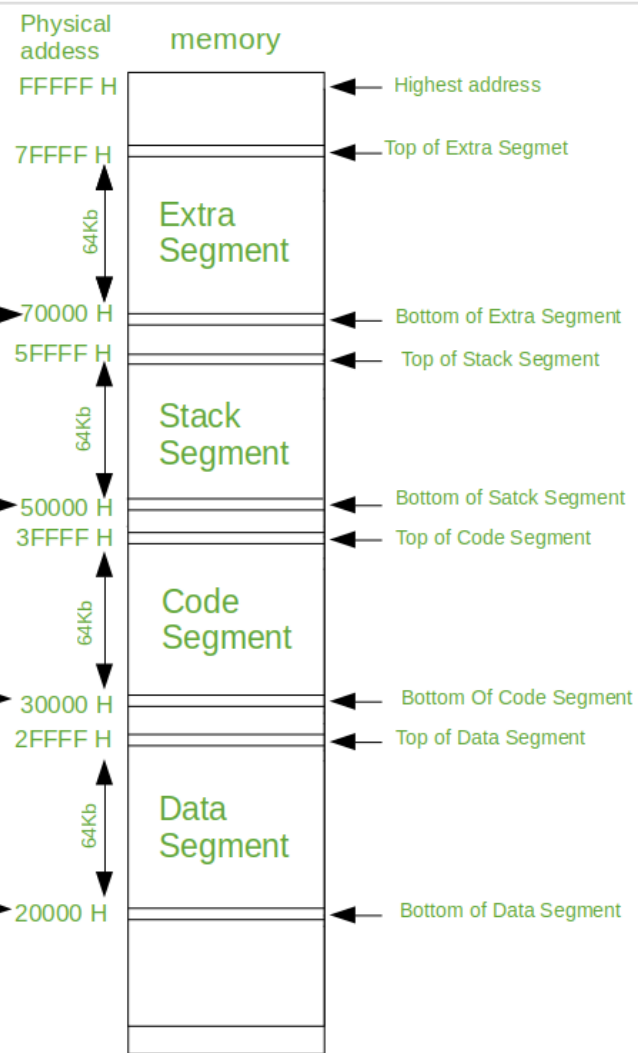
DS (Data Segment register): holds the starting address of the data segment where the program's variables and arrays are located.

ES (Extra Segment register): can be used as an additional data segment register for certain memory operations, such as string operations.

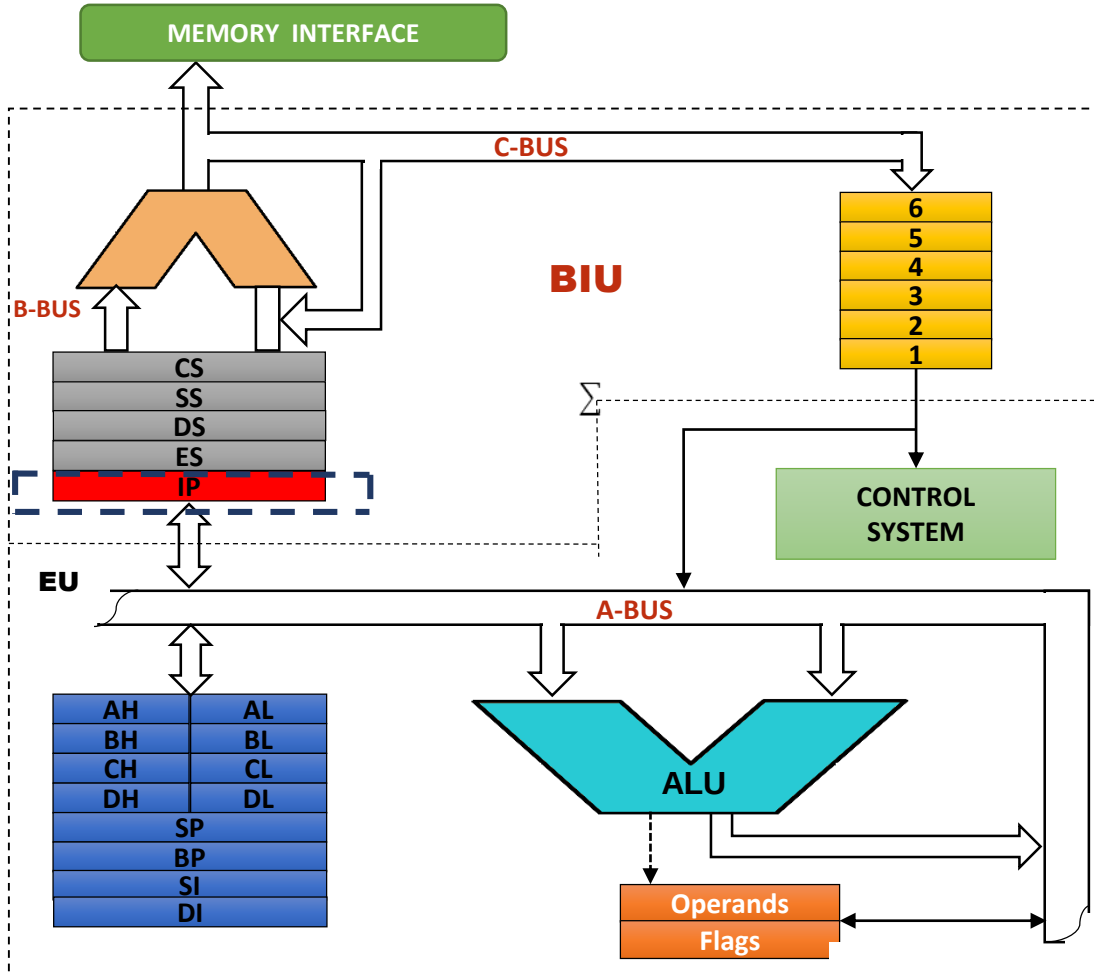
Four segment registers
In BIU

ES	7	0	0	0
CS	3	0	0	0
SS	5	0	0	0
DS	2	0	0	0

Segment registers hold the upper 16 bits of the starting addresses of four memory segments that 8086 is working with at any particular time.

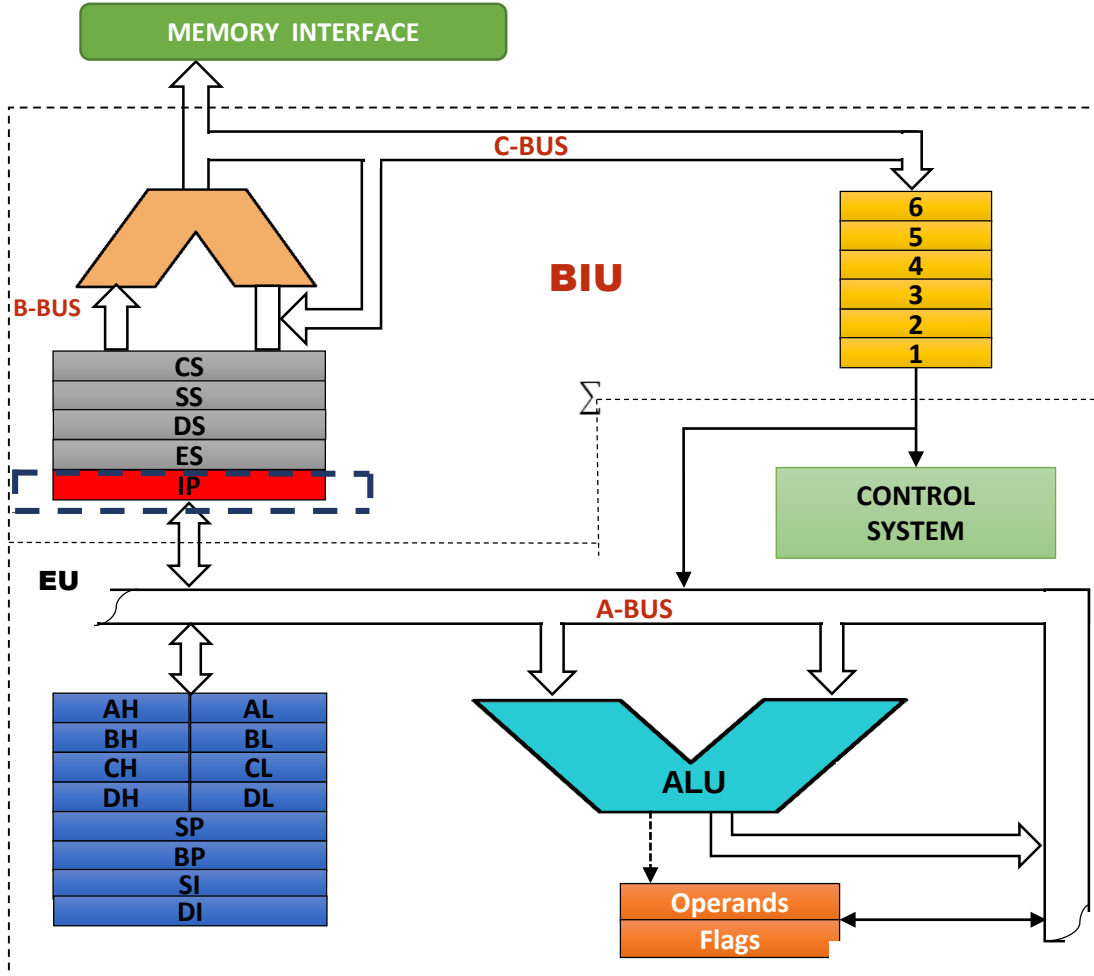


Bus Interface Unit (BIU)



- **16-bit Register:** Points to the memory offset of the **next instruction** to be executed.
- **CS:IP Pairing:** Works with the **Code Segment (CS)** register to locate instructions in memory.
- **Segment & Offset:** **CS** holds the segment base address; **IP** holds the specific offset within that segment.
- **Automatic Updates:** Increments automatically after each fetch or via control transfers

Bus Interface Unit (BIU)



The IP in the Instruction Cycle

Fetch: The BIU uses the current CS:IP to fetch an instruction byte from memory.

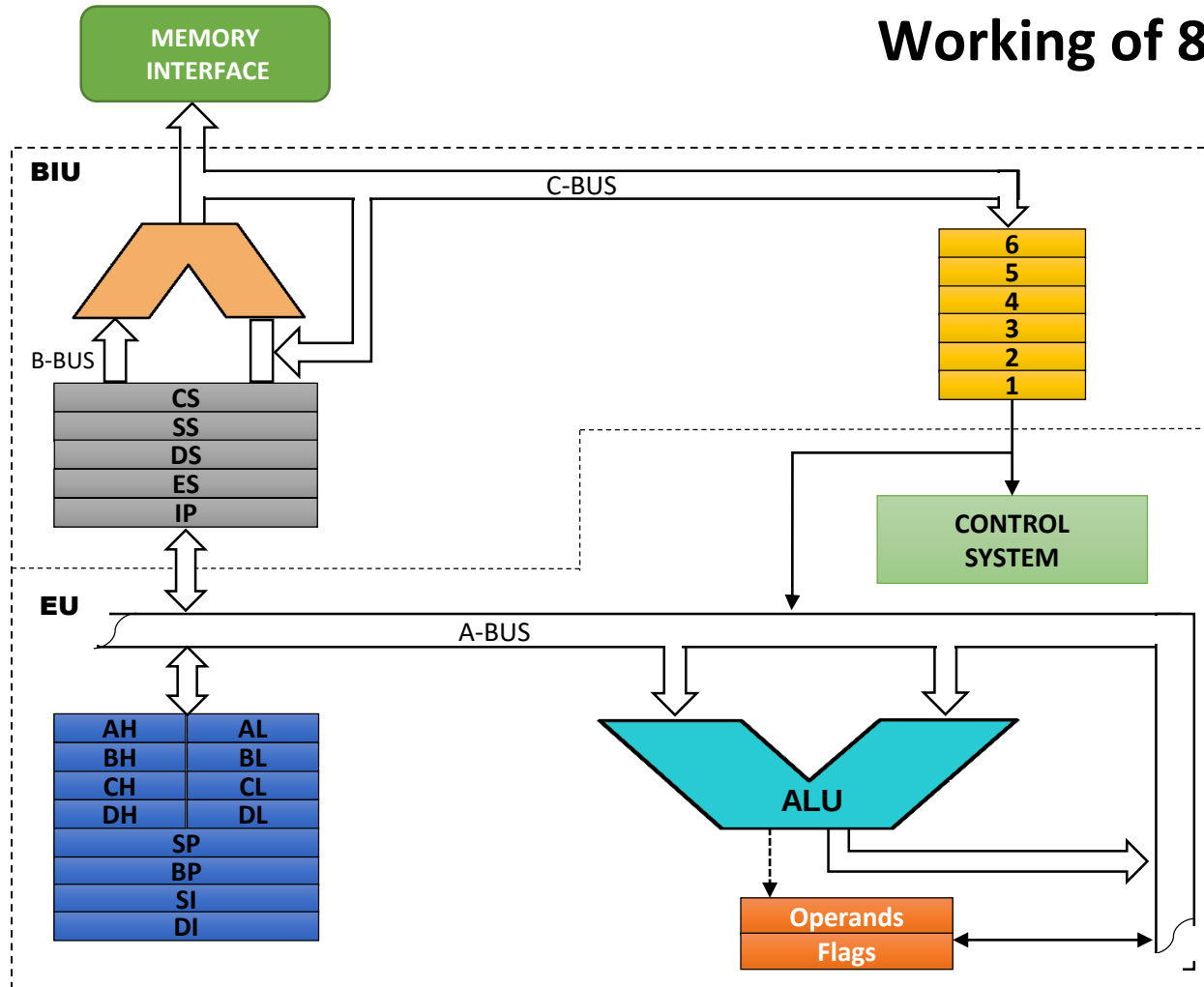
Increment: As soon as a byte is fetched, the **IP is incremented** to point to the next byte.

Execute: The Execution Unit (EU) processes the instruction.

Repeat: The cycle continues.

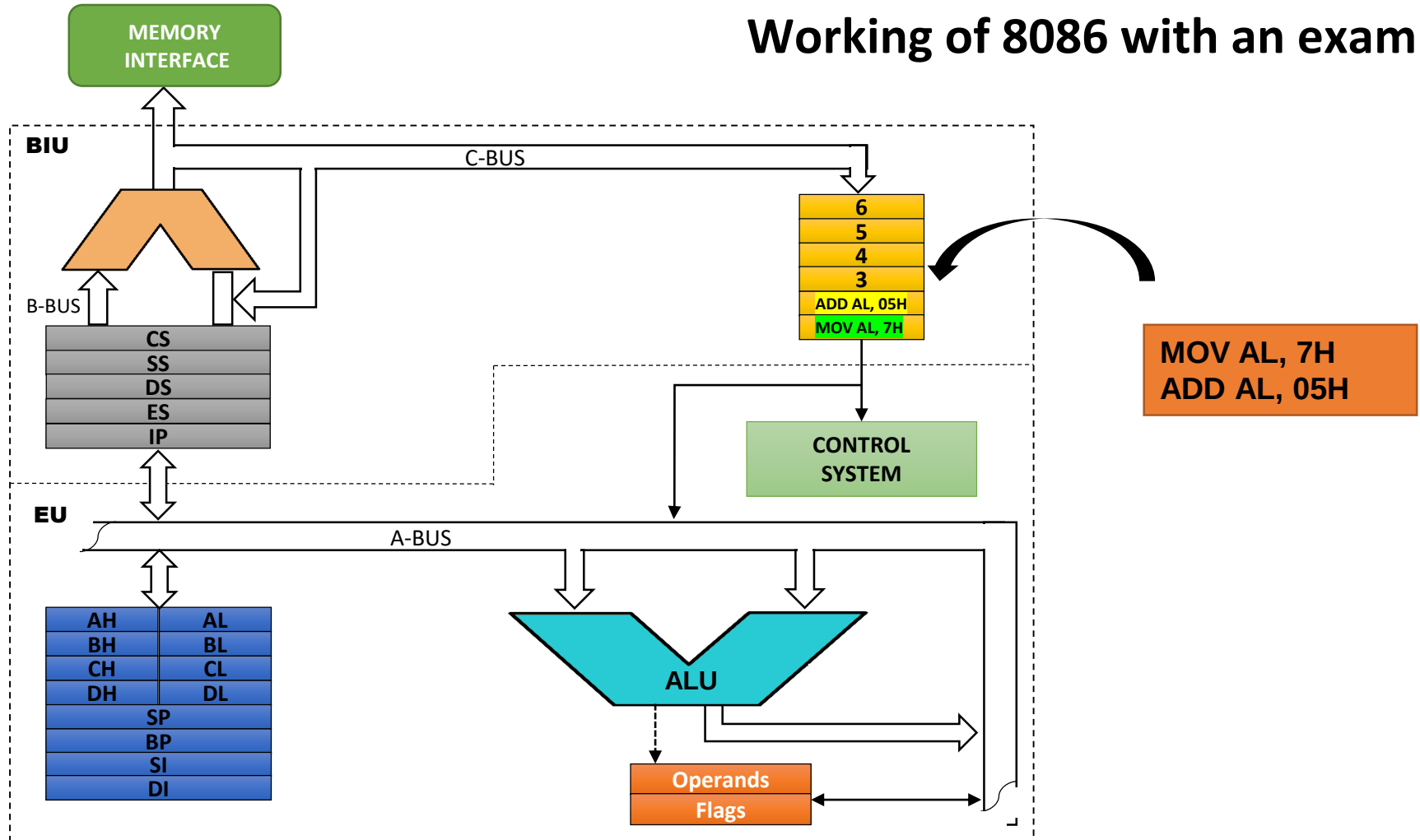
Note: The IP always points to the *next* instruction, not the one currently being executed.

Working of 8086 with an example

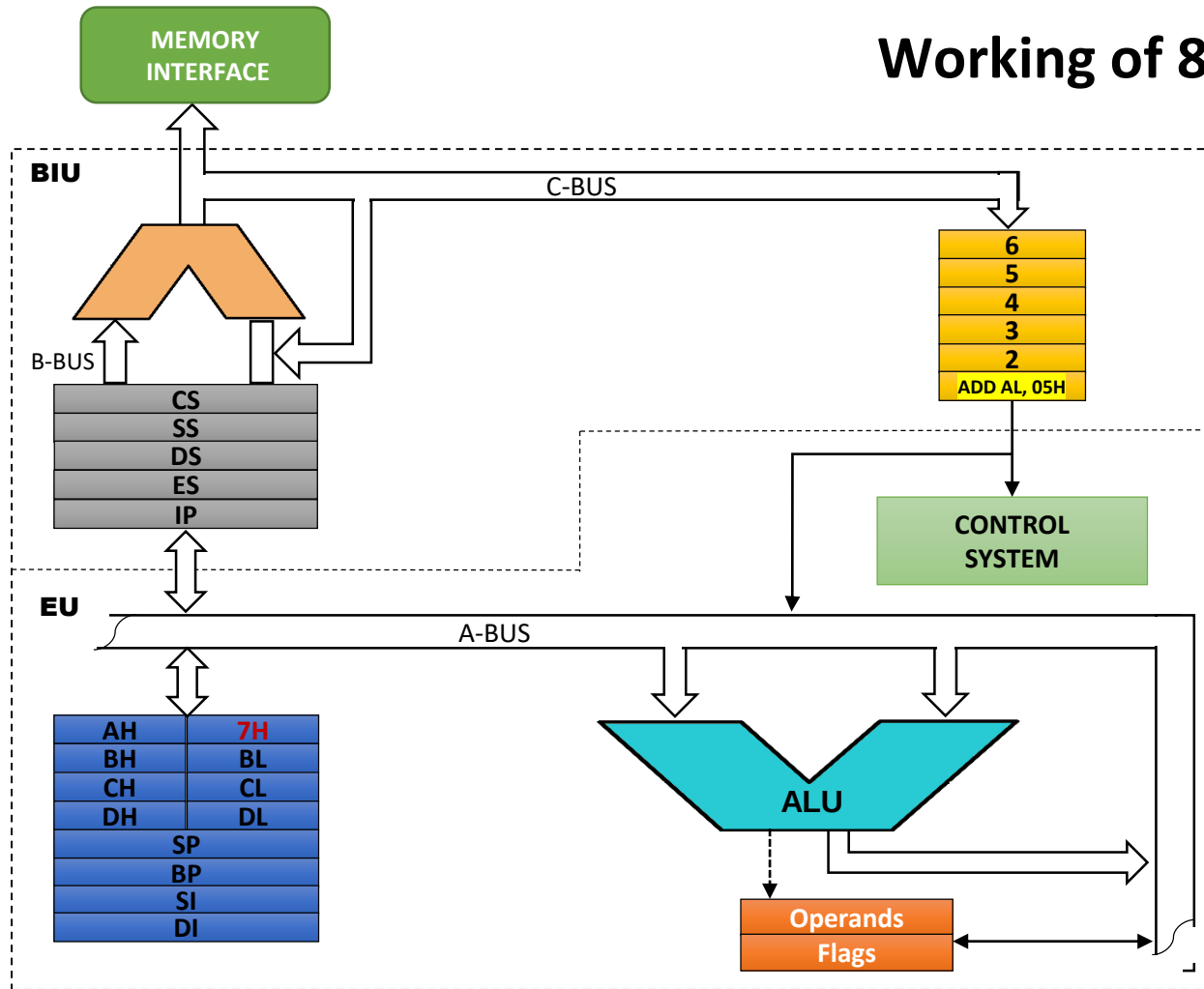


```
MOV AL, 7H  
ADD AL, 05H
```

Working of 8086 with an example

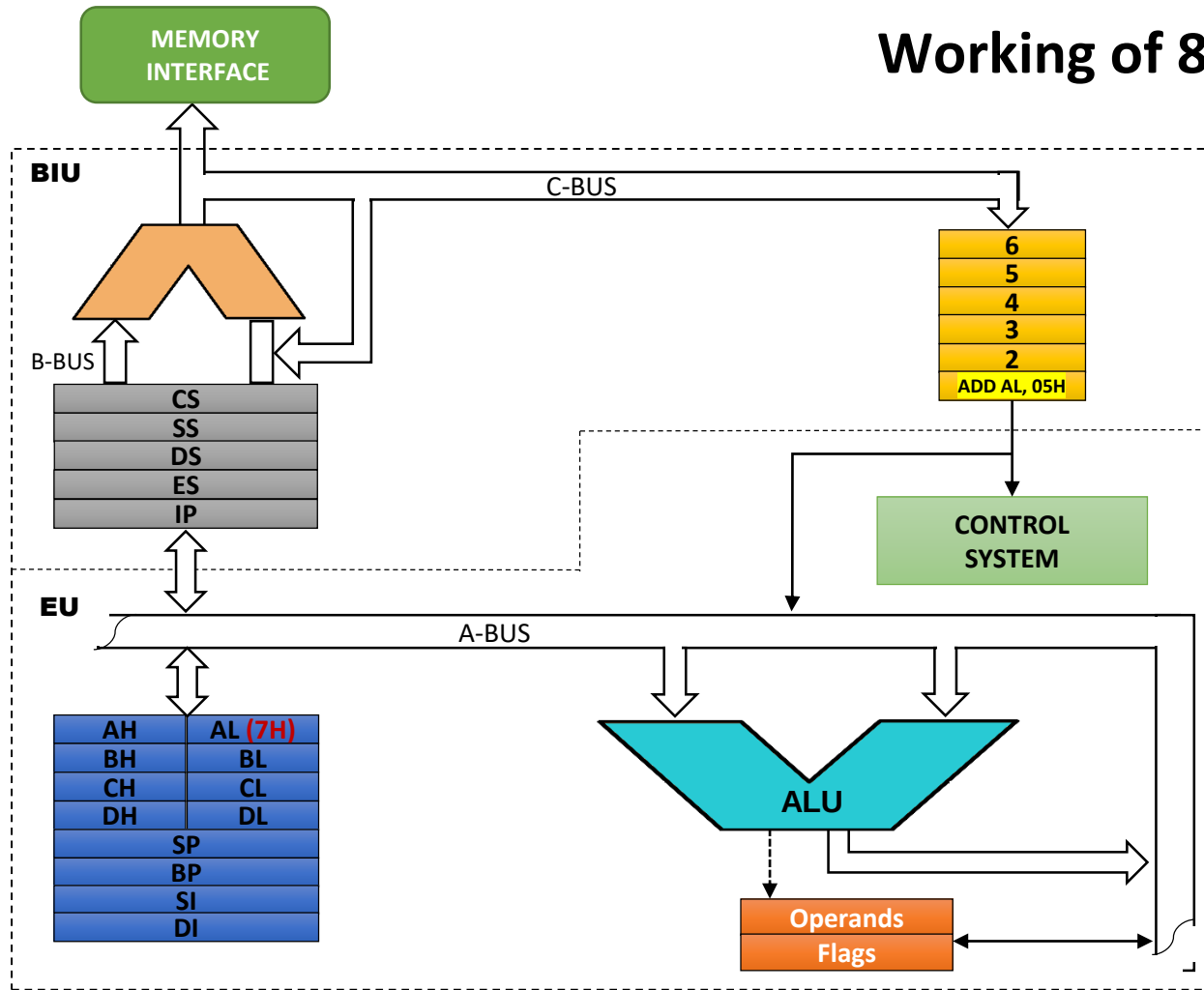


Working of 8086 with an example



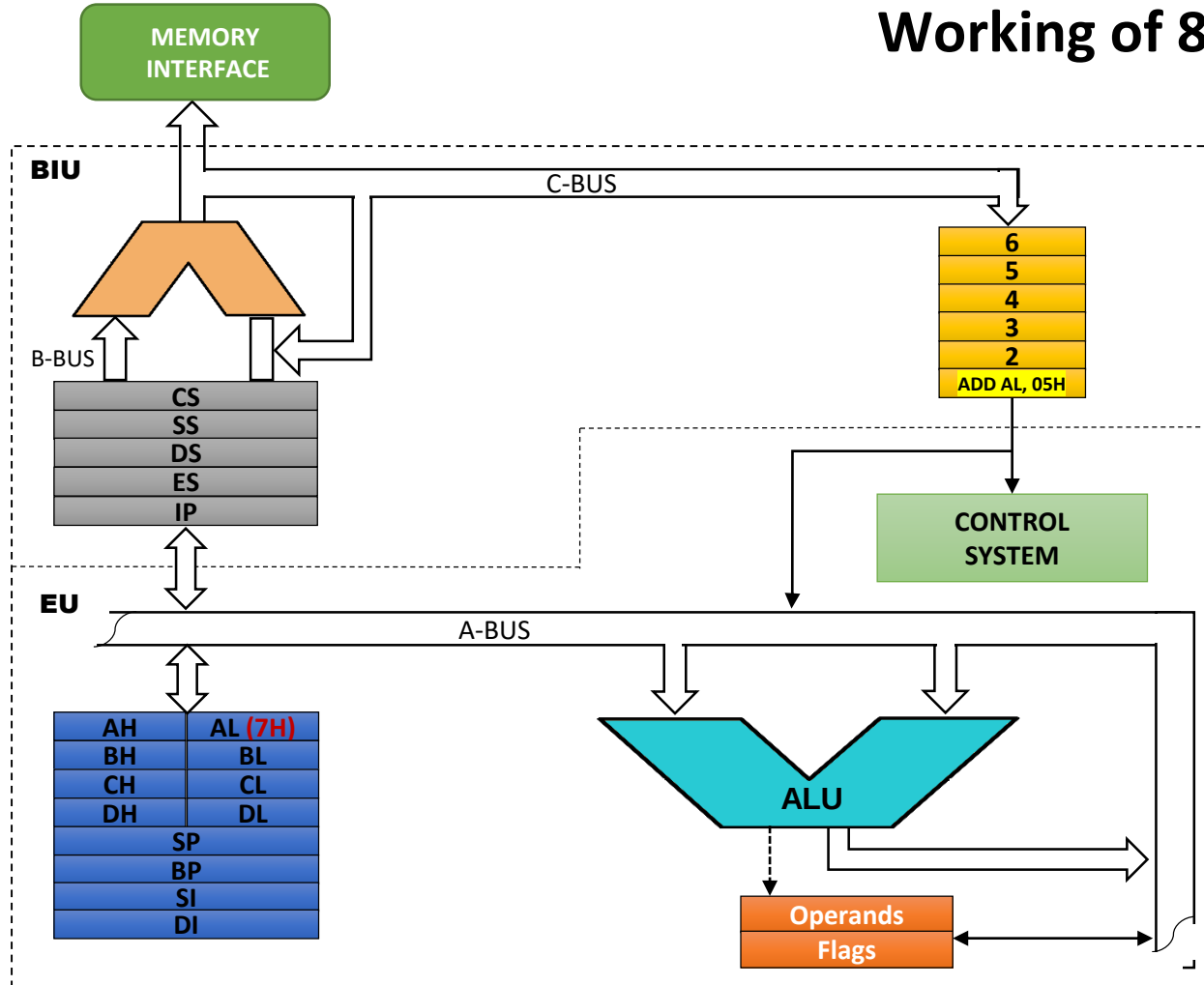
MOV AL, 7H
ADD AL, 05H

Working of 8086 with an example



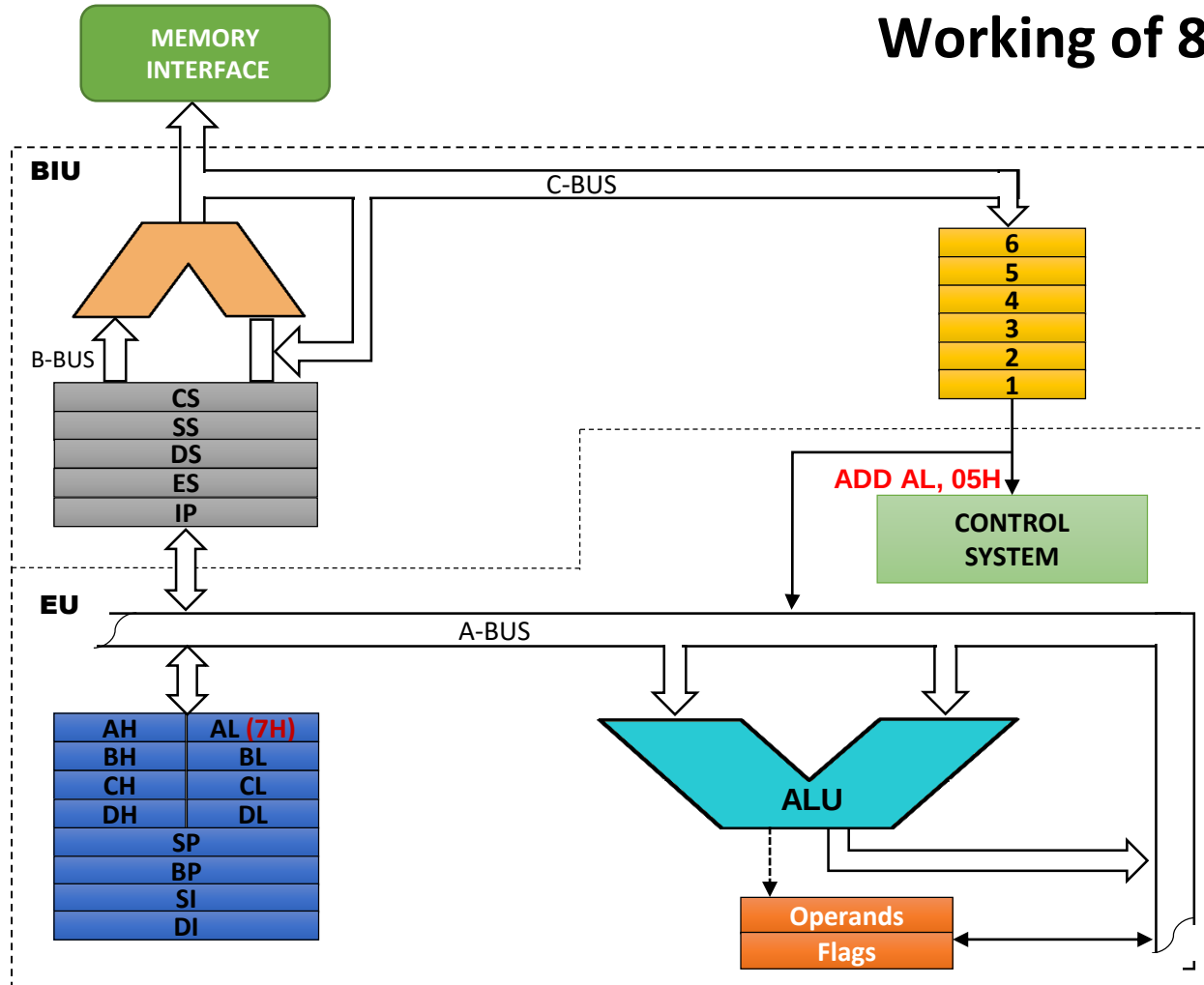
MOV AL, 7H
ADD AL, 05H

Working of 8086 with an example



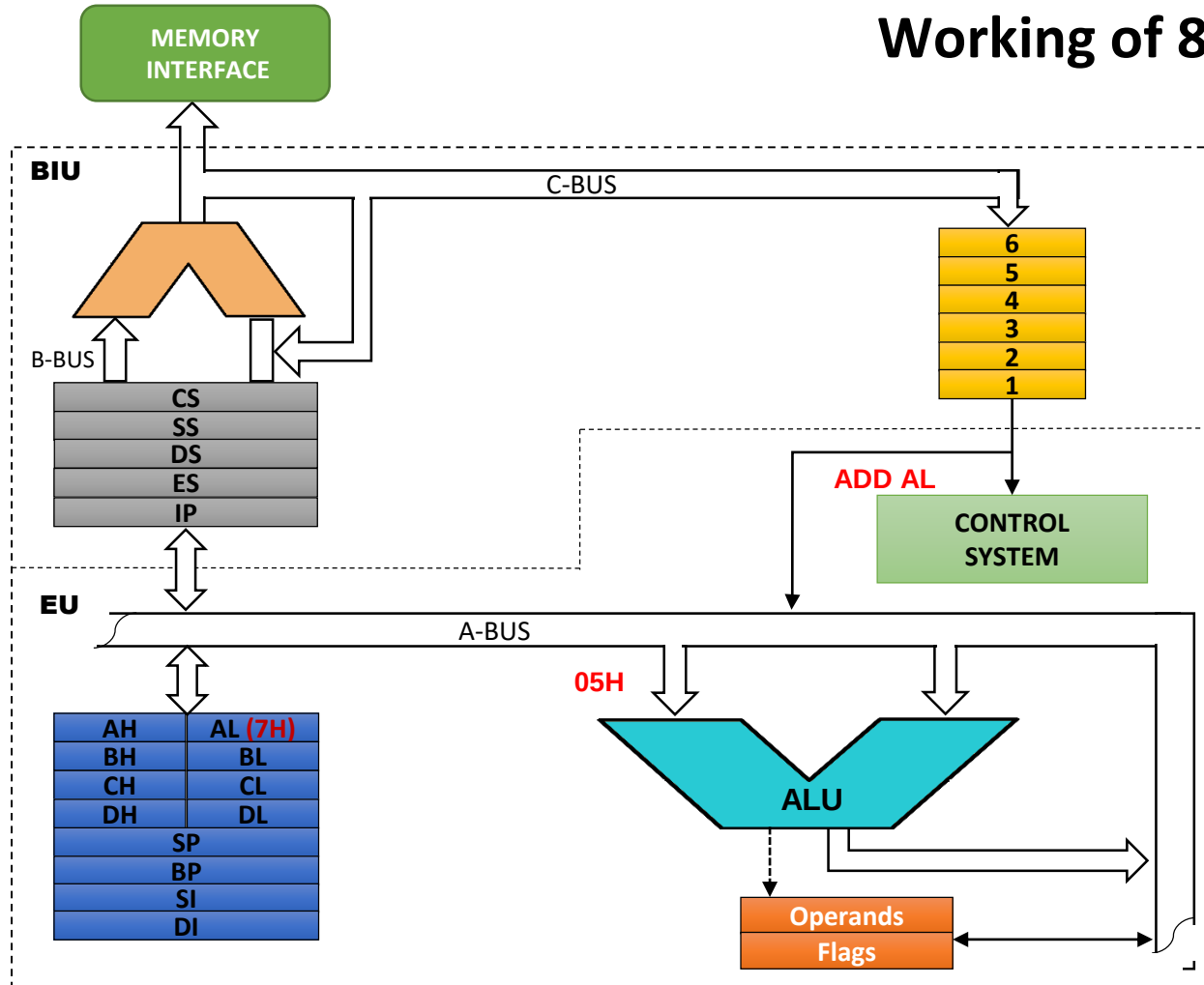
MOV AL, 7H
ADD AL, 05H

Working of 8086 with an example



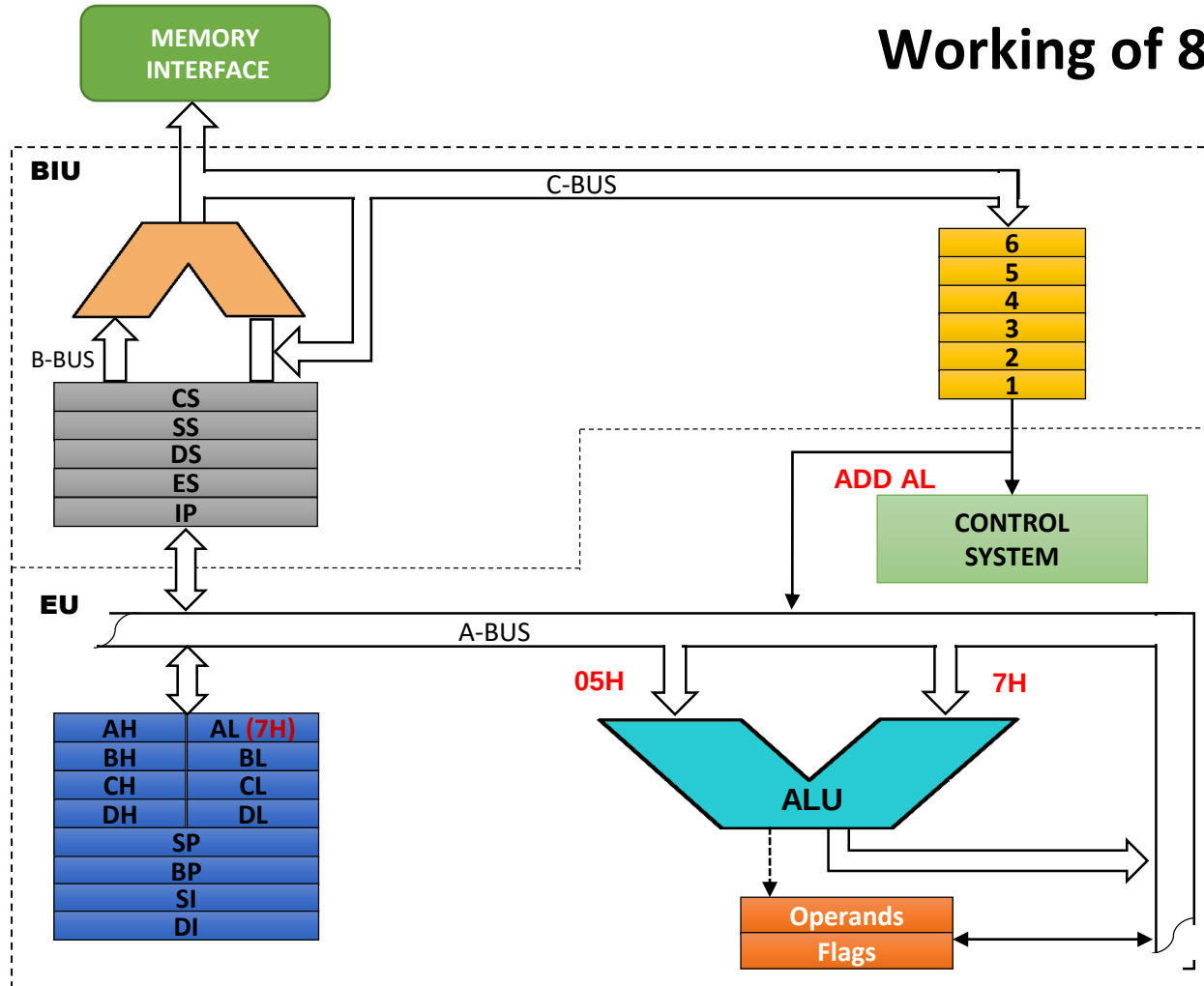
MOV AL, 7H
ADD AL, 05H

Working of 8086 with an example

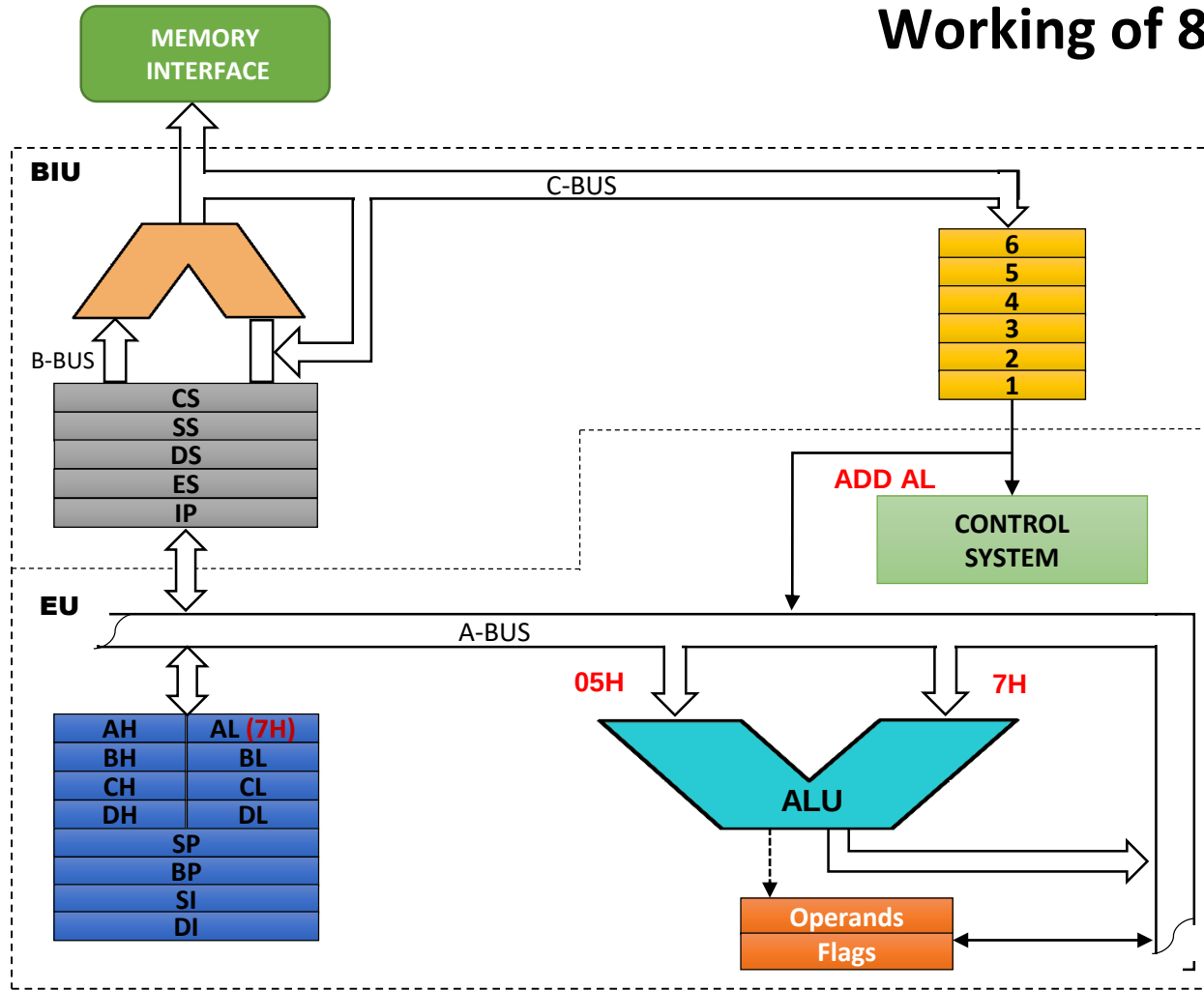


MOV AL, 7H
ADD AL, 05H

Working of 8086 with an example

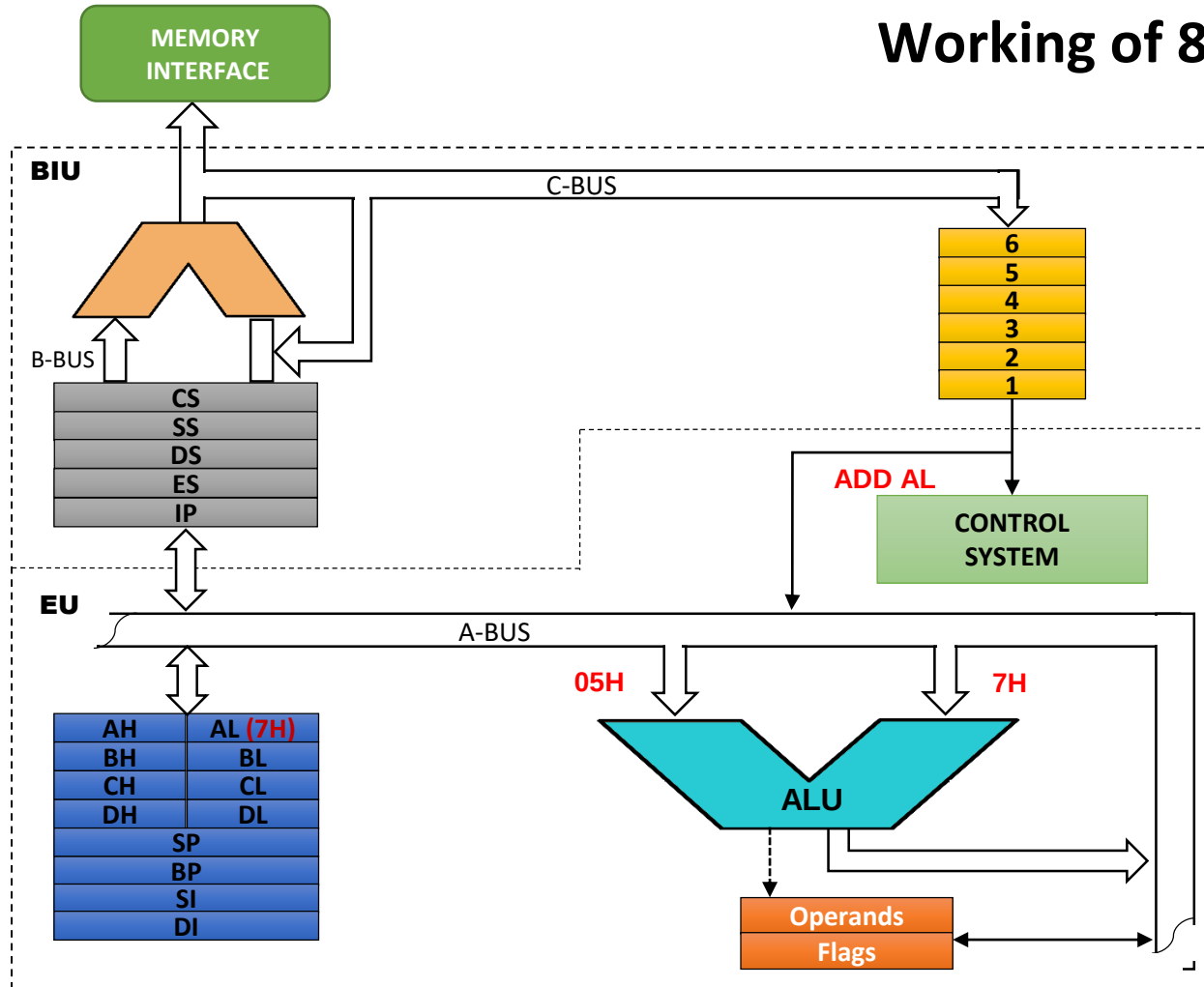


Working of 8086 with an example



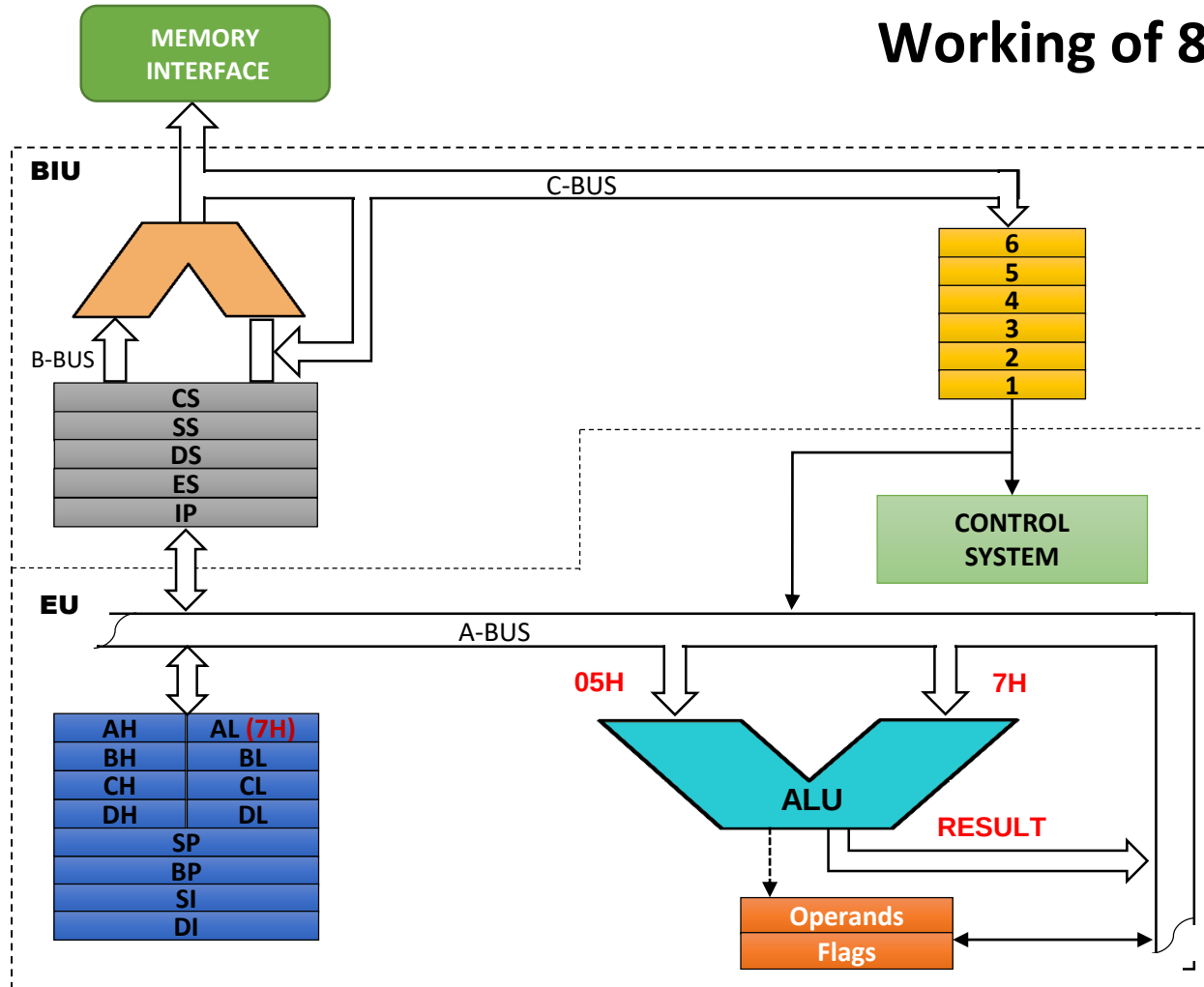
MOV AL, 7H
ADD AL, 05H

Working of 8086 with an example



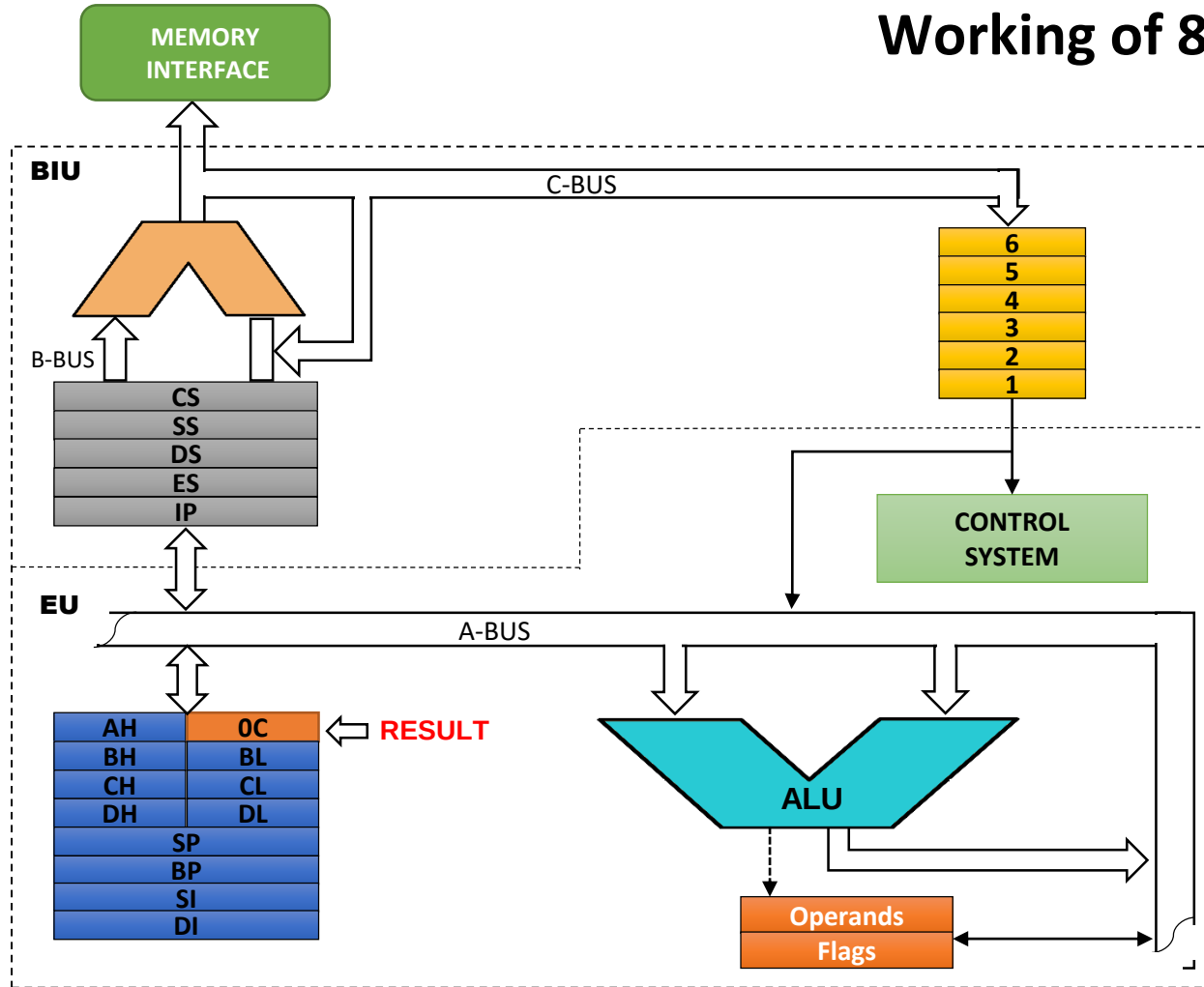
MOV AL, 7H
ADD AL, 05H

Working of 8086 with an example



MOV AL, 7H
ADD AL, 05H

Working of 8086 with an example



MOV AL, 7H
ADD AL, 05H



BITS Pilani
K K Birla Goa Campus

THANK YOU

